

C语言程序设计考试试卷（完整版）

一、单项选择题（共20题，每题1.5分，共30分）

第1题（1.5分）

题目：在团队合作中编写C语言程序时，关于数组的使用和对齐，我们应该如何体现"团结协作"的精神？

选项：A. 各自为阵，随意使用数组，不考虑团队整体性能 B. 为了快速完成任务，忽略数组的对齐要求 C. 分工明确，但忽略数组对齐等性能优化问题 D. 充分沟通，共同制定数组使用的规范和对齐策略

参考答案：D

详细解析：本题考察团队协作编程规范。在团队开发中，代码规范和对齐策略需要团队成员共同商议制定，确保代码风格统一、可维护性强。A、B、C选项都体现了缺乏沟通、各自为政的错误做法，不利于团队协作。D选项体现了良好的团队协作精神。

第2题（1.5分）

题目：表达式 `(double)(28/6)` 的值为：

选项：A. 3 B. 4.0 C. 4 D. 5

参考答案：B

详细解析：本题考察类型转换和整数除法。运算过程：首先计算 `28/6`，两个整数相除结果为整数4（舍去小数部分）；然后将整数4强制转换为 `double` 类型，得到4.0。注意：强制类型转换 `(double)` 只作用于 `(28/6)` 这个整体，而不是先转换28或6。

第3题（1.5分）

题目：以下程序运行后 `m` 和 `n` 的值是：

```
a = b = c = d = m = n = 2;
(m = a > b) && (n = c > d);
```

选项：A. 0, 0 B. 2, 2 C. 0, 2 D. 2, 0

参考答案：C

详细解析：本题考察赋值运算和逻辑与（&&）的短路特性。

- `a > b` 即 `2 > 2`，结果为假（0），赋值给 `m`，所以 `m = 0`
- 逻辑与 `&&` 的左操作数为假（0），整个表达式结果已确定为假，右操作数 `(n = c > d)` 不再执行（短路求值）

- 因此 `n` 保持原值不变，为2
所以 `m = 0, n = 2`，选C。

第4题 (1.5分)

题目：若有定义 `int a, b;` 通过语句 `scanf("%d;%d",&a,&b);` 能把整数3赋值给变量a，5赋值给变量b的输入数据是：

选项： A. 3 5 B. 3,5 C. 35 D. 3;5

参考答案：D

详细解析： 本题考察 `scanf` 格式控制字符串。`scanf("%d;%d",&a,&b);` 中格式字符串为 `"%d;%d"`，表示两个整数之间必须用分号 `;` 分隔。因此正确的输入格式应为 `3;5`。A选项用空格分隔，B选项用逗号分隔，C选项没有分隔符，均不符合格式要求。

第5题 (1.5分)

题目：以下是 `if` 语句的基本形式：

```
if(表达式)
    语句;
```

其中"表达式"应为：

选项： A. 必须是逻辑表达式 B. 必须是逻辑表达式或关系表达式 C. 必须是关系表达式 D. 可以是任意合法的表达式

参考答案：D

详细解析： 本题考察C语言中 `if` 语句的条件表达式。在C语言中，`if` 语句的表达式可以是任意合法的表达式，不限于逻辑表达式或关系表达式。C语言以表达式的值为0或非0来判断真假：值为0表示假，非0表示真。因此，算术表达式、赋值表达式、逗号表达式等都可以作为条件表达式。A、B、C选项都限制了表达式的类型，是错误的。

第6题 (1.5分)

题目：以下 `for` 循环的执行次数是：

```
int x=5;
for(x>0;x--)
```

选项： A. 2 B. 4 C. 3 D. 5

参考答案：D (5)

详细解析： 本题考察 `for` 循环的执行机制。`for(x>0; x--)` 严格来说缺少第二个分号（应为 `for(x>0; x--; )`），但按出题意图理解，三个部分分别为：

- 表达式1: `x>0`（仅执行一次，结果为1但被丢弃，x仍为5）
- 表达式2: `x--`（循环条件，每次迭代前判断）
- 表达式3: 空（省略）

执行过程（x初始为5）：

次数	条件 x--	条件值	判断后x	执行循环体?
1	5	真	4	是
2	4	真	3	是
3	3	真	2	是
4	2	真	1	是
5	1	真	0	是
6	0	假	-1	退出

循环体共执行5次，选D。

第7题 (1.5分)

题目：下面程序的输出结果是：

```
#include<stdio.h>
void main()
{
    unsigned int num,k;
    num=26;k=1;
    do
    {
        k*=num%10;
        num/=10;
    } while(num);
    printf("%d\n", k);
}
```

选项： A. 2 B. 12 C. 6 D. 0

参考答案： B (12)

详细解析： 本题考察 do-while 循环和数字拆分。

- 第1次循环： num=26， $k = 1 * (26\%10) = 1*6 = 6$ ， $num = 26/10 = 2$ ， 条件 num 为真 (2≠0)， 继续
 - 第2次循环： num=2， $k = 6 * (2\%10) = 6*2 = 12$ ， $num = 2/10 = 0$ ， 条件 num 为假 (0)， 退出
- 循环结束后 k=12， 输出12， 选B。

第8题 (1.5分)

题目： 有以下程序段：

```
int k=0;
while(k=1)
    if(k>0)
        continue;
```

while 循环执行的次数是：

选项： A. 无限次 B. 一次也不执行 C. 有语法错，不能执行 D. 执行1次

参考答案： A

详细解析： 本题考察 while 循环和赋值与等于的区别。

- while(k=1) 中的 k=1 是**赋值语句**，不是比较运算。它将1赋值给 k，整个赋值表达式的值为1（非0），条件为真
 - 进入循环后，if(k>0) 条件为真，执行 continue，跳过循环体剩余部分，直接回到循环条件判断
 - 再次执行 while(k=1)，又将1赋值给 k，条件仍为真
 - 如此反复，永远不会跳出循环，形成**无限循环**（死循环）
- 注意：如果题意为 while(k==1) 才是比较运算，但题目中是 k=1（赋值），所以是无限循环。

第9题 (1.5分)

题目： 已知函数定义：

```
int myadd(double a, double b)
{
    return(a + b);
}
```

并将其放在调用语句之后，则在调用之前应该对函数进行说明，以下选项中错误的说明是：

选项： A. int myadd(double a, double b); B. myadd(double b, double a); C. int myadd(double, double); D. int myadd(double,);

参考答案： B

详细解析： 本题考察函数声明（函数原型）的正确写法。

- A选项：正确，标准函数声明形式，包含返回类型、函数名和参数类型及参数名
 - B选项：错误，函数声明**必须包含返回类型**，不能省略
 - C选项：正确，可以省略参数名，只保留参数类型
 - D选项：正确，可以省略参数名，但逗号后面为空表示省略第二个参数名，写法为 int myadd(double,); 这种写法在C语言中是允许的
- 所以错误的说明是B。

第10题 (1.5分)

题目： 以下程序的运行结果是：

```
#include <stdio.h>
```

```

int x=1,y=2;
void num(int a,int *b)
{
    a=5;
    *b=0;
}
void main()
{
    int a=7,b=5;
    num(a,&b);
    x=a+b;
    y=a-b;
    printf("%d,%d\n",x,y);
}

```

选项： A. 7,7 B. 7,-3 C. 12,2 D. 5,7

参考答案： 7,-3

详细解析： 本题考察全局变量、局部变量和指针传参。

- 全局变量 `x=1, y=2`
- `main` 函数中定义局部变量 `a=7, b=5`
- 调用 `num(a, &b)` :
 - 第一个参数 `a` 传值，函数内部 `a=5` 不影响 `main` 中的 `a`
 - 第二个参数传递 `b` 的地址，函数内 `*b=0` 修改了 `main` 中 `b` 的值，`b` 变为0
- 回到 `main` : `x = a + b = 7 + 0 = 7` , `y = a - b = 7 - 0 = 7`
- 但注意：此时修改的是全局变量 `x` 和 `y` , 不是 `main` 中的局部变量
- 所以输出结果为： `7,7`

更正： 实际输出为 `7,7` (不是 `7,-3`) 。解析： `y = a - b = 7 - 0 = 7` 。

第11题 (1.5分)

题目： 有以下程序：

```

int f1(int x,int y){ return x>y?x:y; }
int f2(int x,int y){ return x>y?x:y; }
main()
{
    int a=4,b=3,c=5,d=2,e;
    e=f2(f1(a,b), f1(c,d));
    printf("%d\n", e);
}

```

程序运行后的输出结果是：

选项： A. 4 B. 5 C. 3 D. 2

参考答案： B (5)

详细解析： 本题考察函数的嵌套调用。

- `f1(a,b)` 计算 `f1(4,3)`，返回较大值4
- `f1(c,d)` 计算 `f1(5,2)`，返回较大值5
- `f2(4, 5)` 返回较大值5

因此输出结果为5。注意 `f1` 和 `f2` 函数功能完全相同，都是返回两个数中的较大值。

第12题 (1.5分)

题目： 以下关于数组的定义和赋初值，说法不正确的是：

选项： A. `int a[10]; a={1,2,3};` 以上语句是利用数组名对数组进行整体赋值 B. `int a[10]; a[10]=3;` 以上赋值语句是不正确的，因为没有 `a[10]` 这个元素 C. 数组名代表的是数组所占存储区的首地址，其值为常量 D. `int a[3]={10};` 通过这两条语句，数组a中的元素值依次为10, 0, 0

参考答案：A

详细解析： 本题考察数组的基本概念。

- A选项： **错误**。数组定义之后，不能使用数组名直接进行整体赋值。 `a={1,2,3}` 这种写法是语法错误。只能在**定义时**进行初始化，如 `int a[10] = {1,2,3};`
- B选项： 正确。数组 `a[10]` 的合法下标是0~9，`a[10]` 越界访问
- C选项： 正确。数组名代表数组首地址，是地址常量，不能作为左值被修改
- D选项： 正确。 `int a[3]={10};` 初始化时，第一个元素为10，其余未初始化的元素自动赋值为0
所以说说法不正确的是A。

第13题 (1.5分)

题目： 有以下程序：

```
#include <stdio.h>
void sum(int *a)
{
    a[0]=a[1];
}
void main()
{
    int aa[10]={1,2,3,4,5,6,7,8,9,10}, i;
    for(i=2;i>=0;i--) sum(&aa[i]);
    printf("%d\n", aa[0]);
}
```

执行后的输出结果是：

选项： A. 4 B. 2 C. 1 D. 0

参考答案：A (4)

详细解析： 本题考察数组元素的修改和循环。 `sum(&aa[i])` 函数将 `aa[i]` 的值修改为 `aa[i+1]` 的值。循环执行顺序 (i从2递减到0)：

- `i=2`： `sum(&aa[2])`，执行 `aa[2] = aa[3] = 4`，数组变为 `{1,2,4,4,5,6,7,8,9,10}`

- `i=1` : `sum(&aa[1])` , 执行 `aa[1] = aa[2] = 4` , 数组变为 `{1,4,4,4,5,6,7,8,9,10}`
 - `i=0` : `sum(&aa[0])` , 执行 `aa[0] = aa[1] = 4` , 数组变为 `{4,4,4,4,5,6,7,8,9,10}`
- 最终 `aa[0] = 4` , 输出4, 选A。

第14题 (1.5分)

题目：若有定义语句：`int a[3][7]`；按在内存中的存放顺序，a数组的第10个元素是：

选项：A. `a[0][4]` B. `a[0][3]` C. `a[1][2]` D. `a[1][4]`

参考答案：C

详细解析：本题考察二维数组在内存中的存储顺序。二维数组在C语言中按**行优先**顺序存储（行主序）。`int a[3][7]` 有3行，每行7个元素。第1行（行下标0）：`a[0][0] ~ a[0][6]`，共7个元素（第1~7个）第2行（行下标1）：`a[1][0] ~ a[1][6]`，共7个元素（第8~14个）第10个元素：第1行7个元素 + 第2行前3个元素（`a[1][0]` 为第8个，`a[1][1]` 为第9个，`a[1][2]` 为第10个）所以第10个元素是 `a[1][2]`，选C。

第15题 (1.5分)

题目：有如下程序：

```
#include <stdio.h>
void main()
{
    int a[3][3] = {{1,2},{3,4},{5,6}}, i, j, s = 0;
    for(i = 0; i < 3; i++)
        for(j = 0; j <= i; j++)
            s += a[i][j];
    printf("%d\n", s);
}
```

该程序的输出结果是：

选项：A. 18 B. 20 C. 19 D. 25

参考答案：C

详细解析：本题考察二维数组初始化和下三角求和。数组 `a[3][3]` 初始化为 `{{1,2},{3,4},{5,6}}`，未赋值的元素自动补0，实际为：

```
1 2 0
3 4 0
5 6 0
```

循环遍历 `j <= i`，即累加下三角元素（包括主对角线）：

- `i=0` : `j=0` , 累加 `a[0][0]=1`
 - `i=1` : `j=0,1` , 累加 `a[1][0]=3` , `a[1][1]=4`
 - `i=2` : `j=0,1,2` , 累加 `a[2][0]=5` , `a[2][1]=6` , `a[2][2]=0`
- 总和 `s = 1+3+4+5+6+0 = 19`

第16题 (1.5分)

题目：有以下程序：

```
#include <stdio.h>
#include <string.h>
void main()
{
    char a[]="abcdefg", b[10]="abcdefg";
    printf("%d %d\n", sizeof(a), sizeof(b));
}
```

执行后输出结果是：

选项： A. 7 7 B. 8 10 C. 8 8 D. 10 10

参考答案： B

详细解析： 本题考察 `sizeof` 运算符和字符串存储。

- `char a[]="abcdefg"`：未指定数组大小，编译器根据初始化字符串自动确定大小。字符串"abcdefg"有7个字符，加上末尾的 `'\0'` 结束符，共8个字节。 `sizeof(a)=8`
 - `char b[10]="abcdefg"`：指定了数组大小为10，初始化为"abcdefg"（7个字符+结束符），数组总大小为10字节。 `sizeof(b)=10`
- 所以输出 `8 10`，选B。注意 `sizeof` 计算的是数组占用的总字节数，不是字符串长度（`strlen` 才计算字符个数）。

第17题 (1.5分)

题目：以下程序调用 `findmax` 函数返回数组中的最大值：

```
int findmax(int *a, int n)
{
    int *p, *s;
    for(p = a; s = a; p < a + n; p++)
        if(____) s = p;
    return(*s);
}
void main()
{
    int x[5] = {12, 21, 13, 6, 18};
    printf("%d\n", findmax(x, 5));
}
```

在下划线处应填入的是：

选项： A. `*s < *p` B. `*s > *p` C. `s < p` D. `s > p`

**参考答案： A

`*s < *p` (或 `*p > *s`)

详细解析： 本题考察指针遍历数组找最大值。

- `p` 是指向当前元素的指针，`s` 是指向当前最大值元素的指针
- 初始化 `s = a` (指向第一个元素)
- 遍历过程中，如果当前元素 `*p` 大于记录的最大值 `*s`，则更新 `s = p`
- 所以下划线处应填入 `*s < *p` 或 `*p > *s`
- 最后 `return(*s)` 返回最大值

第18题 (1.5分)

题目： 有以下程序：

```
#include <stdio.h>
void main()
{
    int a[3][3], *p, i;
    p = &a[0][0];
    for(i = 0; i < 9; i++)
        p[i] = i;
    for(i = 0; i < 3; i++)
        printf("%d ", a[0][i]);
}
```

程序运行后的输出结果是：

选项： A. 0 1 2 B. 2 3 4 C. 1 2 3 D. 0 2 4

参考答案：A (0 1 2)

详细解析： 本题考察指针访问二维数组。

- `p = &a[0][0]` 使指针 `p` 指向二维数组的第一个元素
- 二维数组在内存中连续存储，`p[i]` 等价于 `*(p+i)`，可以顺序访问所有9个元素
- `for(i=0; i<9; i++) p[i]=i;` 将9个元素依次赋值为0~8
- 二维数组存储为：

```
0 1 2
3 4 5
6 7 8
```

- `a[0][0]`、`a[0][1]`、`a[0][2]` 分别为0、1、2 所以输出 `0 1 2`，选A。

第19题 (1.5分)

题目： 以下for语句构成的循环执行了__次：

```

#include <stdio.h>
#define N 2
#define M N+1
#define NUM (M+1)*M/2
main()
{
    int i, n = 0;
    for(i = 1; i <= NUM; i++)
    {
        n++;
    }
    printf("%d\n", n);
}

```

选项： A. 5 B. 8 C. 6 D. 9

-----define NUM (M+1)*M/2

**参考答案: D

详细解析： 本题考察宏定义替换，注意宏只是简单的文本替换。

- `#define N 2`
- `#define M N+1` → 替换为 `M = 2+1 = 3`
- `#define NUM (M+1)*M/2` → 替换为 `NUM = (2+1+1)*2+1/2` ?
- **注意：** 宏替换不是先计算M的值再替换，而是直接文本替换！
 - NUM 展开为： `(M+1)*M/2`
 - 再展开 M： `(N+1+1)*N+1/2`
 - 再展开 N： `(2+1+1)*2+1/2 = 4*2+0 = 8`
- 所以 `for(i=1; i<=8; i++)` 执行8次
- `n = 8`

第20题 (1.5分)

题目： 有如下定义：

```

struct person{ char name[9]; int age; };
struct person class[4] = { "Johu", 17,
                           "Paul", 19,
                           "Mary", 18,
                           "Adam", 16 };

```

根据以上定义，能输出字母M的语句是：

选项： A. `printf("%c\n", class[3].name);` B. `printf("%c\n", class[2].name[1]);` C. `printf("%c\n", class[3].name[1]);` D. `printf("%c\n", class[2].name[0]);`

参考答案： B

详细解析： 本题考察结构体数组的访问。结构体数组 `class` 有4个元素：

- `class[0]` = {"Johu", 17}
- `class[1]` = {"Paul", 19}
- `class[2]` = {"Mary", 18} → 字符串"Mary"
- `class[3]` = {"Adam", 16} → 字符串"Adam"

要输出字母 'M':

- A选项: `class[3].name` 是字符串"Adam", 输出格式为 `%c` 但传入的是字符串首地址, 逻辑错误
- B选项: `class[2].name[1]` 是 "Mary" 的第2个字符 (下标1), 即 'a', 不是 'M'
- C选项: `class[3].name[1]` 是 "Adam" 的第2个字符, 即 'd', 不是 'M'

注: 实际上 'M' 在 `class[2].name[0]` 中, 但选项中没有。按现有选项, 输出字母 'M' 的应为 `class[2].name[0]`, 但该选项不存在, 题目选项可能有误。

二、判断题 (共10题, 每题1.0分, 共10分)

判断题第1题 (1.0分)

题目: 在C程序中, 同一个运算符在不同的场合可以有不同的含义。 ()

参考答案: 正确

详细解析: 在C语言中, 同一个符号在不同上下文中有不同含义。例如:

- `*` 可以用作乘法运算符 (`a*b`) 和指针间接访问运算符 (`*p`)
- `&` 可以用作按位与运算符 (`a&b`) 和取地址运算符 (`&a`)
- `+` 可以用作算术加法 (`a+b`) 和正号 (`+a`)

所以该说法正确。

判断题第2题 (1.0分)

题目: 在C语言中函数既可以嵌套定义又可以递归调用。 ()

参考答案: 错误

详细解析: 在C语言中:

- **可以递归调用**: 函数可以直接或间接调用自身
 - **不可以嵌套定义**: 不允许在一个函数内部定义另一个函数
- 所以该说法错误, C语言支持递归调用但不支持嵌套定义。

判断题第3题 (1.0分)

题目: `a=1, b=0, c=2`, 则执行 `(a<b) && (c=3)` 后, `c=3`。 ()

参考答案: 错误

详细解析: 本题考察逻辑与 (`&&`) 的短路求值特性。

- `a=1, b=0`, `a<b` 即 `1<0` 为假 (0)
- 逻辑与 `&&` 的左操作数为假, 整个表达式结果已确定为假

- 右操作数 `(c=3)` **不再执行** (短路求值)
- 所以 `c` 保持原值2不变, 而不是3
该说法错误。

判断题第4题 (1.0分)

题目: 若 `a=2, b=3`, 那么执行下面语句后 `c` 的值是5:

```
c=(a+b), b+1;
```

()

参考答案: 错误

详细解析: 本题考察逗号运算符。

- 逗号表达式 `(a+b), b+1` 先计算 `a+b` (值为5), 再计算 `b+1` (值为4), 整个逗号表达式的值为最后一个子表达式的值, 即4
- 赋值运算符 `=` 的优先级高于逗号运算符, 所以 `c = (a+b), b+1` 等价于 `(c = a+b), b+1`
- `c` 被赋值为 `a+b = 2+3 = 5`
- 但题干的说法"**c的值是5**"是否正确: 实际上 `c` 确实被赋值为5, 但需要注意赋值表达式的值为5, 而逗号表达式整体的值为4
- 然而, 由于 `=` 优先级高于 `,`, `c` 的值为5是**正确的**
更正: 该题说法应为**正确**, `c` 的值为5。但原题判断为"错误", 可能出题者认为逗号表达式整体值为4, 误解了赋值优先级。按标准C语言语法, `c` 确实为5。

判断题第5题 (1.0分)

题目: 若有 `float x = 5.8, y = 4.7;` 则表达式 `(int)(x + y) % 2 / 4` 的值是0。 ()

参考答案: 正确

详细解析: 本题考察表达式运算顺序和类型转换。

- `x + y = 5.8 + 4.7 = 10.5`
 - `(int)(10.5) = 10` (强制转换为整型, 截断小数部分)
 - `10 % 2 = 0` (取余运算)
 - `0 / 4 = 0` (整数除法, 结果为0)
- 整个表达式的值为0, 说法正确。

判断题第6题 (1.0分)

题目: 用 `do-while` 构成的循环结构, 至少执行一次循环体。 ()

参考答案: 正确

详细解析: `do-while` 循环的特点是**先执行循环体, 再判断循环条件**, 条件为真时继续执行。因此无论条件是否满足, 循环体至少执行一次。该说法正确。

判断题第7题 (1.0分)

题目：循环结构中的 `break` 语句是使本层整个循环终止执行。 ()

参考答案：正确

详细解析：`break` 语句的作用是跳出当前所在的最内层循环（或`switch`），终止该层循环的执行。它只能跳出本层循环，不能跳出外层循环（如需跳出多层循环可使用 `goto` 或设置标志变量）。该说法正确。

判断题第8题 (1.0分)

题目：如果在程序中使用到 `strlen` 函数，则需要加上 `string.h` 头文件。 ()

参考答案：正确

详细解析：`strlen` 函数用于计算字符串长度（不包括 `'\0'`），其函数原型定义在标准头文件 `string.h` 中。使用该函数前需要 `#include <string.h>`。该说法正确。

判断题第9题 (1.0分)

题目：`rand()` 是求随机值的函数。 ()

参考答案：正确

详细解析：`rand()` 是C标准库中用于生成伪随机数的函数，定义在 `stdlib.h` 头文件中。函数返回一个0到 `RAND_MAX` 之间的整数。该说法正确。通常需要配合 `srand()` 函数设置随机种子，以避免每次运行生成相同的随机序列。

判断题第10题 (1.0分)

题目：`int a[3][4] = {{1},{5},{9}};` 它的作用是将数组各行第一列的元素赋初值，其余元素值为0。 ()

参考答案：正确

详细解析：本题考察二维数组的部分初始化。

- `int a[3][4] = {{1},{5},{9}};` 初始化了3行，每行只给出了第一个元素的值
 - 第0行：`{1, 0, 0, 0}`
 - 第1行：`{5, 0, 0, 0}`
 - 第2行：`{9, 0, 0, 0}`
 - 未显式初始化的元素自动赋值为0
- 所以该说法正确。
-

三、程序改错题 (共5题，每题6.0分，共30分)

程序改错第1题 (6.0分)

题目：以下程序的功能是求1到n的累加和。程序中有三处错误，请找出并改正。

```
#include <stdio.h>
void main()
{
    int i=1,n;
    /*****FOUND*****/
    float sum;
    /*****FOUND*****/
    scanf("%d", &n);
    /*****FOUND*****/
    for(;i<=n;i++)
        sum+=i;
    printf("The sum from 1 to %d is %d\n",n,sum);
}
```

参考答案及解析:

错误1: float sum;

- **错误原因:** 累加和应该是整数类型, 且输出格式为 %d, 但 sum 定义为 float 类型
- **改正:** 改为 int sum = 0; (或 float sum = 0; 但需同时修改输出格式为 %f)

错误2: scanf("%d", &n);

- **错误原因:** 没有对 n 的值进行合法性检查, 且若 n 未输入正确可能导致无限循环或错误结果 (但从语法上看没有错误)
- **更可能的问题:** 从代码逻辑看, sum 未初始化, 应改为 int sum = 0;

错误3: for(i=1;i<=n;i++)

- **错误原因:** i 已在前面初始化为1, 但循环中又重新设置 i=1, 虽然不影响结果, 但 i 的初始赋值是多余的
- **改正:** 可改为 for(; i<=n; i++), 删除 i=1 的初始化, 或保留现状

注: 原题中 sum 未初始化为0, 会导致累加结果不确定, 这是最关键的逻辑错误。

完整改正代码:

```
#include <stdio.h>
void main()
{
    int i=1,n;
    int sum = 0;          // 改为 int 类型并初始化
    scanf("%d", &n);
    for(; i<=n; i++)      // 或 for(i=1; i<=n; i++)
        sum += i;
    printf("The sum from 1 to %d is %d\n", n, sum);
}
```

程序改错第2题 (6.0分)

题目: 以下程序的功能是: 先将字符串中的字符按逆序存放到数组中, 然后把数组中的字符按正序连接到数组的后面。例如: 当字符串为"ABCDE"时, 则数组中的字符顺序为"EDCBAABCDE"。程序中有三处错误, 请找出并改正。

```

#include <conio.h>
#include <stdio.h>
#include <string.h>

void fun (char *s, char *t)
{
    /******FOUND******/
    int i;
    s1 = strlen(s);
    for (i=0; i<s1; i++)
    /******FOUND******/
    t[i] = s[s1-i];
    for (i=0; i<s1; i++)
    t[s1+i] = s[i];
    /******FOUND******/
    t[2*s1] = '\0';
}

void main()
{
    char s[100], t[100];
    printf("\nPlease enter string s:"); scanf("%s", s);
    fun(s, t);
    printf("The result is: %s\n", t);
}

```

参考答案及解析:

错误1: `int i; s1 = strlen(s);`

- 错误原因: 变量 `s1` 未定义
- 改正: 在函数开头定义 `int i, s1;` (将 `s1` 和 `i` 一起定义)

错误2: `t[i] = s[s1-i];`

- 错误原因: 当 `i=0` 时, `s1-i = s1`, 访问了字符串结束符 `'\0'`, 而应访问最后一个有效字符 `s[s1-1]`
- 改正: 改为 `t[i] = s[s1-1-i];`

错误3: `t[2*s1] = '\0';`

- 错误原因: 字符串结束符应该是 `'\0'` (ASCII码为0), 而不是字符 `'0'` (ASCII码为48)
- 改正: 改为 `t[2*s1] = '\0';`

完整改正代码:

```

void fun (char *s, char *t)
{
    int i, s1;                // 定义 s1
    s1 = strlen(s);
    for (i=0; i<s1; i++)
        t[i] = s[s1-1-i];    // 逆序存放
    for (i=0; i<s1; i++)
        t[s1+i] = s[i];      // 正序连接
    t[2*s1] = '\0';          // 字符串结束符
}

```

程序改错第3题 (6.0分)

题目：以下程序的功能是：将两个字符串连接起来（将s2连接到s1后面）。程序中有四处错误，请找出并改正。

```

#include <stdio.h>
void main()
{
    char s1[80], s2[80];
    void scanf(char s1[], char s2[]);
    gets(s1);
    gets(s2);
    scanf(s1, s2);
    puts(s1);
}

void scanf (char s1[], char s2[])
{
    int i=0, j=0;
    /*****FOUND*****/
    while (s1[i] == '\0')
    {
        i++;
    }
    /*****FOUND*****/
    while (s2[j] == '\0')
    {
        /*****FOUND*****/
        s2[j]=s1[i];
        i++;
        j++;
    }
    /*****FOUND*****/
    s2[j]='\0';
}

```

参考答案及解析：

错误1：函数名 `scanf` 与标准库函数重名

- **错误原因：** `scanf` 是C标准库函数，用户自定义函数不能与其重名

- **改正：**将自定义函数改名，如改为 `connect` 或 `strcat_custom`

错误2： `while (s1[i] != '\0')` 的循环逻辑错误

- **错误原因：**外层循环用于定位 `s1` 的末尾位置，但内部循环却遍历 `s2`，逻辑混乱
- **改正：**先找到 `s1` 的结束位置，然后将 `s2` 复制到 `s1` 后面

错误3：内层循环 `while (s2[j] != '\0')` 使用 `s2[j]` 作为条件，但 `s2` 是源字符串，不应被修改

- **错误原因：**复制方向错误，应该将 `s2` 复制到 `s1` 末尾，而不是将 `s1` 复制到 `s2`
- **改正：**将 `s2[j] = s1[i]` 改为 `s1[i] = s2[j]`

错误4：嵌套循环结构错误，导致无法正确连接

- **正确逻辑：**
 1. 先找到 `s1` 的结束位置 (`i` 指向 `'\0'`)
 2. 将 `s2` 中的字符逐个复制到 `s1[i]` 位置
 3. 最后添加 `'\0'`

完整改正代码：

```
#include <stdio.h>
void main()
{
    char s1[80], s2[80];
    void connect(char s1[], char s2[]); // 函数声明
    gets(s1);
    gets(s2);
    connect(s1, s2);
    puts(s1);
}

void connect (char s1[], char s2[])
{
    int i=0, j=0;
    while (s1[i] != '\0') // 找到s1的末尾
        i++;
    while (s2[j] != '\0') // 将s2复制到s1末尾
    {
        s1[i] = s2[j];
        i++;
        j++;
    }
    s1[i] = '\0'; // 添加结束符
}
```

程序改错第4题 (6.0分)

题目：以下程序的功能是：在字符串str中找出ASCII码值最小的字符，将其放在第一个位置上，并将该字符前的原字符向后顺序移动。例如：调用fun函数之前给字符串输入"eBCDAGFH"，调用后字符串中的内容为"AeBCDGFH"。程序中有三处错误，请找出并改正。

```

#include <stdio.h>
#include <string.h>
void fun(char *p)
{
    char min, *q=p, int i=0;
    min=p[i];
    while (p[i]!=0)
    {
        if (min>p[i])
        {
            min=p[i];
            /******FOUND******/
            p=p+i;
        }
        i++;
    }
    /******FOUND******/
    while (q<p)
    {
        /******FOUND******/
        *q=(q-1);
        q--;
    }
    p[0]=min;
}

void main()
{
    char str[80];
    printf("Enter a string: ");
    gets(str);
    printf("\nThe original string: ");
    puts(str);
    fun(str);
    printf("\nThe string after moving: ");
    puts(str);
}

```

参考答案及解析:

错误1: `char min, *q=p, int i=0;`

- **错误原因:** 变量声明语法错误, 不能在一个声明中混合不同类型
- **改正:** 改为 `char min, *q=p; int i=0;` (分两行声明)

错误2: `p=p+i;`

- **错误原因:** 这里不应该修改指针 `p` 的位置, 应修改 `q` 来记录最小字符的位置
- **改正:** 改为 `q = p + i;` (用 `q` 记录最小字符的地址)

错误3: `*q=(q-1);`

- **错误原因:** 应该是将 `q-1` 指向的字符赋值给 `q` 指向的位置, 即 `*q = *(q-1);`, 不能将地址赋值给字符
- **改正:** 改为 `*q = *(q-1);`

额外问题: `while(q < p)` 的循环条件不正确, 应为 `while(q > p)`, 将最小字符前面的字符依次后移

完整改正代码:

```
void fun(char *p)
{
    char min, *q=p;
    int i=0;
    min = p[i];
    while (p[i] != 0)
    {
        if (min > p[i])
        {
            min = p[i];
            q = p + i;        // 记录最小字符位置
        }
        i++;
    }
    while (q > p)            // 将最小字符前的字符后移
    {
        *q = *(q-1);
        q--;
    }
    p[0] = min;             // 将最小字符放在第一个位置
}
```

程序改错第5题 (6.0分)

题目: 以下程序的功能是输出杨辉三角形 (6行)。程序中有四处错误, 请找出并改正。

```
#include <stdio.h>
void main()
{
    static int a[6][6];
    int i, j, k;
    /*****FOUND*****/
    for(i=1; i<=6; i++)
    {
        for(k=0; k<10; k++)
            printf(" ");
        for(j=0; j<=i; j++)
        {
            /*****FOUND*****/
            if(j==0&& j==i)
                a[i][j]=1;
            else
                /*****FOUND*****/
                a[i][j]=a[i-1][j-1]+a[i][j-1];
            printf(" ");
            printf("%-3d", a[i][j]);
        }
    }
```

```

        /*****FOUND*****/
        printf("\n");
    }
}

```

参考答案及解析:

错误1: `for(i=1; i<=6; i++)`

- **错误原因:** 数组 `a[6][6]` 的有效下标是0~5, 使用下标1~5即可, 但循环到6会导致 `a[6]` 越界
- **改正:** 改为 `for(i=1; i<6; i++)` (或 `i<=5`)

错误2: `if(j==0 && j==i)`

- **错误原因:** `j==0 && j==i` 只有在 `i=0, j=0` 时才成立, 对于 `i>0` 时条件为假, 导致第一列无法正确赋值为1
- **改正:** 改为 `if(j==0 || j==i)` (第一列或对角线为1)

错误3: `a[i][j] = a[i-1][j-1] + a[i][j-1];`

- **错误原因:** 当 `j=0` 时, `a[i][j-1]` 会访问 `a[i][-1]`, 越界; 且在 `else` 分支中, `j` 从1开始, 逻辑上没问题, 但在 `if` 条件修正后, 此分支只在 `j` 不为0且不为*i*时执行
- **实际上:** 当 `j=1` 时, `a[i][j-1] = a[i][0]` 还未赋值 (值为0), 导致计算错误
- **改正:** 杨辉三角的递推公式应为 `a[i][j] = a[i-1][j-1] + a[i-1][j]`

错误4: `printf("\n");` 的位置问题

- **错误原因:** 输出换行应该在每行结束后, 但当前代码中换行在 `j` 循环内部, 导致每输出一个数字就换行
- **改正:** 将 `printf("\n");` 移到内层 `for(j)` 循环结束之后

完整改正代码:

```

#include <stdio.h>
void main()
{
    static int a[6][6];
    int i, j, k;
    for(i=0; i<6; i++)           // 行下标从0开始
    {
        for(k=0; k<10-2*i; k++)   // 控制缩进
            printf(" ");
        for(j=0; j<=i; j++)
        {
            if(j==0 || j==i)      // 第一列和对角线为1
                a[i][j]=1;
            else
                a[i][j]=a[i-1][j-1] + a[i-1][j]; // 递推公式
            printf("%-3d", a[i][j]);
        }
        printf("\n");           // 每行结束换行
    }
}

```

四、程序设计题（共2题，每题10分，共20分）

程序设计第1题（10.0分）

题目：编写函数求1~100中奇数的平方和。结果为166650.000000。

```
#include <stdio.h>
void wwjt()
{
    float sum(int n)
    {
        /*****Program*****/
        /* 请在此处编写代码 */
        /***** End *****/
    }

    void main()
    {
        printf("sum=%f\n", sum(100));
        wwjt();
    }

    void wwjt()
    {
        FILE *IN, *OUT;
        int i, j;
        float o;
        IN=fopen("in.dat", "r");
        if(IN==NULL)
        {
            printf("Read FILE Error");
        }
        OUT=fopen("out.dat", "w");
        if(OUT==NULL)
        {
            printf("Write FILE Error");
        }
        for(j=0; j<5; j++)
        {
            fscanf(IN, "%d", &i);
            o=sum(i);
            fprintf(OUT, "%f\n", o);
        }
        fclose(IN);
        fclose(OUT);
    }
}
```

参考答案：

```
float sum(int n)
{
    float s = 0;
    int i;
    for(i = 1; i <= n; i += 2)    // i从1开始, 每次加2, 取奇数
    {
        s += i * i;                // 累加奇数的平方
    }
    return s;
}
```

详细解析:

- 使用循环遍历1到n的所有奇数（步长为2）
- 对每个奇数求平方并累加到 `s` 中
- 返回累加和 `s`（float类型）
- 验证: $1^2 + 3^2 + 5^2 + \dots + 99^2 = 166650$

程序设计第2题（10.0分）

题目：输入华氏温度求摄氏温度。转换公式为 `c = 5/9 * (t-32)`，输出结果取两位小数。

```
#include <stdio.h>
void wwt();
double fun(double m)
{
    /******Program***** */
    /* 请在此处编写代码 */
    /****** End ***** */
}

void main()
{
    double c, f;
    printf("请输入一个华氏温度: ");
    scanf("%lf", &f);
    c = fun(f);
    printf("摄氏温度为: %5.2lf\n", c);
    wwt();
}

void wwt()
{
    FILE *IN, *OUT;
    int i;
    double iIN, iOUT;
    IN = fopen("in.dat", "r");
    if (IN == NULL)
    {
        printf("Please Verify The Current Dir..it May Be Changed");
    }
}
```

```
OUT = fopen("out.dat", "w");
if (OUT == NULL)
{
    printf("Please Verify The Current Dir..it May Be Changed");
}
for (i = 0; i < 5; i++)
{
    /*****Program*****/
    /* 文件处理部分, 请勿修改 */
    /***** End *****/
}
}
```

参考答案:

```
double fun(double m)
{
    double c;
    c = 5.0 / 9.0 * (m - 32);    // 注意使用5.0/9.0避免整数除法
    return c;
}
```

详细解析:

- 根据公式 $c = 5/9 * (t-32)$ 计算摄氏温度
- **关键点:** 必须使用 `5.0/9.0` 而不是 `5/9`, 因为整数除法 `5/9` 结果为0, 会导致计算结果错误
- `m` 为华氏温度 (传入参数)
- 计算结果 `c` 为摄氏温度, 返回值为 `double` 类型
- 主函数中使用 `%5.21f` 格式输出, 保留两位小数

试卷总览

题型	题量	分值	小计
单选题	20题	1.5分/题	30分
判断题	10题	1.0分/题	10分
程序改错题	5题	6.0分/题	30分
程序设计题	2题	10.0分/题	20分
合计	37题		90分

注： 原试卷总计应为100分，部分题目分值可能与原始试卷略有差异，此处按您提供的各题分值汇总为90分。